MSNE Python Function Documentation

Auto-generated

2025-12-12

Contents

1 Introduction **1**

2 generate_msne_jobs.py **1**

2.1 generate_msne_jobs . . . 2

3 run_MSNE_*.py **3**

3.1 set_seeds . . . 3

4 evaluate_msne_results.py **3**

4.1 main . . . 3

5 Alphabetical Index **4**

6 Notes **4**

6.1 MSNE Algorithm Overview . . . 4

6.2 Input Data Format . . . 5

6.3 Key Hyperparameters . . . 5

6.4 Typical Output Structure . . . 5

1 Introduction

This document provides comprehensive documentation for custom functions used in the MSNE (Multiple Similarity Network Embedding) implementation for the MultiOmicsSurvey project. Functions are organized alphabetically within each source file section.

---

2 generate_msne_jobs.py

Hyperparameter grid search job generator for MSNE clustering on HPC systems.

2.1 generate_msne_jobs

```
generate_msne_jobs()
```

Description: Generate MSNE job scripts for hyperparameter grid search on HPC clusters.

Creates Python execution scripts (.py) and SLURM batch submission scripts (.sh) for all combinations of MSNE hyperparameters. This enables systematic exploration of the hyperparameter space for multi-omics clustering optimization.

Parameter Grid:

Parameter	Values	Description
`k` (neighbors)	[10, 20, 30, 40, 50]	Number of nearest neighbors for graph construction
`num_walks`	[50, 100, 200]	Number of random walks per node
`embed_size`	[50, 100, 200]	Dimensionality of embedding space
`window_size`	[5, 10, 15]	Context window for Skip-gram model
`walk_length`	[20, 30, 40]	Length of each random walk

Total Combinations: $5 \times 3 \times 3 \times 3 \times 3 =$ **405 jobs**

Returns: `None` - Files are written to the current working directory.

Side Effects:

- Creates `logs/` directory if it doesn't exist
- Writes 405 `.py` files (run_MSNE_*.py)
- Writes 405 `.sh` files (MSNE_*.sh)
- Sets executable permissions on all generated files

Generated Python Script Features:

- Sets random seeds for reproducibility (seed=123)
- Loads distance matrices from `MO_Dists/` directory
- Runs MSNE with the specific hyperparameter combination
- Saves embeddings and cluster assignments to CSV files

Generated SLURM Script Configuration:

- Partition: `icelake-himem`
- Memory: 200GB
- CPUs: 10 workers
- Time limit: ~12 hours

Example:

```
>>> generate_msne_jobs()
Job scripts generated successfully!
# Creates files like:
# run_MSNE_k_10_nw_50_emb_50_win_5_wl_20.py
# MSNE_k_10_nw_50_emb_50_win_5_wl_20.sh
```

---

3 run_MSNE_*.py

Individual MSNE execution scripts generated by `generate_msne_jobs()`. Each script runs MSNE with a specific hyperparameter combination.

3.1 set_seeds

```
set_seeds(seed)
```

Description: Set random seeds for reproducibility in MSNE clustering.

Ensures deterministic behavior for stochastic operations in the MSNE algorithm by setting identical seeds for NumPy's and Python's random number generators.

Parameters:

Parameter	Type	Description
`seed`	int	The random seed value to set for reproducibility.

Returns: `None`

Example:

```
>>> set_seeds(123)
# All subsequent random operations will be reproducible
```

---

4 evaluate_msne_results.py

Post-processing script for evaluating MSNE hyperparameter search results.

4.1 main

```
main()
```

Description: Evaluate MSNE clustering results and identify optimal hyperparameters.

Scans the current directory for MSNE output files (embeddings and cluster assignments), computes silhouette scores for each hyperparameter combination, and generates a comprehensive evaluation report.

Workflow:

1. Discovers all embedding/cluster file pairs matching `output_*_embeddings.csv`
2. Computes silhouette scores using `sklearn.metrics.silhouette_score`
3. Ranks all hyperparameter combinations by silhouette score
4. Saves detailed results to `all_silhouette_scores.csv`
5. Generates `best_clustering_report.txt` with the optimal configuration

File Naming Convention:

- Embeddings: `output_MSNE_k_{k}_nw_{nw}_emb_{emb}_win_{win}_wl_{wl}_embeddings.csv`
- Clusters: `output_MSNE_k_{k}_nw_{nw}_emb_{emb}_win_{win}_wl_{wl}_clusters.csv`

Returns: `None` - Results are written to files in the current directory.

Output Files:

File	Description
`all_silhouette_scores.csv`	Complete results with columns: job_name, silhouette, n_samples, n_clusters, embeddings_file, clusters_file
`best_clustering_report.txt`	Human-readable summary of optimal hyperparameters

Notes:

- Skips results with fewer than 2 clusters (silhouette undefined)
- Silhouette score ranges from -1 (poor) to 1 (excellent)
- Higher silhouette indicates better-defined cluster separation

Example:

```
>>> main()
Best clustering job: MSNE_k_20_nw_100_emb_100_win_10_wl_30
Silhouette: 0.4523
N samples: 625 N clusters: 5
```

---

5 Alphabetical Index

Function	Source File
generate_msne_jobs	generate_msne_jobs.py
main	evaluate_msne_results.py
set_seeds	run_MSNE_*.py

---

6 Notes

6.1 MSNE Algorithm Overview

MSNE (Multi-view Spectral Network Embedding) performs multi-omics patient clustering by:

1. Loading pre-computed sample-sample distance matrices for each omic view
2. Constructing k-nearest neighbor graphs from each distance matrix
3. Performing random walks on the multi-view graph
4. Learning node embeddings using Skip-gram (Word2Vec-style) approach
5. Clustering samples in the learned embedding space

6.2 Input Data Format

Distance matrices should be stored as CSV files in `MO_Dists/`:

- First column: Sample IDs (index)
- Column headers: Sample IDs
- Values: Pairwise distances between samples

6.3 Key Hyperparameters

Parameter	Description	Impact
`k`	Nearest neighbors	Higher = more connected graph, captures global structure
`num_walks`	Random walks per node	Higher = more training samples for embedding
`embed_size`	Embedding dimensions	Higher = more expressive, risk of overfitting
`window_size`	Context window	Higher = captures longer-range dependencies
`walk_length`	Walk length	Higher = more global context per walk

6.4 Typical Output Structure

```
output_MSNE_k_20_nw_50_emb_100_win_5_wl_20_embeddings.csv  # (625 x 100) embedding matrix
output_MSNE_k_20_nw_50_emb_100_win_5_wl_20_clusters.csv    # (625 x 1) cluster assignments
all_silhouette_scores.csv                                   # Evaluation summary
best_clustering_report.txt                                  # Best hyperparameters
```